



Universidade do Minho

UNIVERSIDADE DO MINHO
MESTRADO EM CIBERSEGURANÇA
2025/2026

Firewall Exploration Lab

PG59783 - Carlos Daniel Silva Fernandes - Mestrado em Cibersegurança
PG59790 - Luís Filipe Pinheiro Silva - Mestrado em Cibersegurança
PG59791 - Pedro Augusto Ennes de Martino Camargo - Mestrado em
Cibersegurança

Questão 1A

Nesta tarefa, foi necessário adicionar um módulo ao kernel do Linux. Para esse efeito, recorreu-se à utilização de um LKM (Loadable Kernel Module), que permite a introdução dinâmica de módulos no kernel em tempo de execução, sem necessidade de recompilar ou reiniciar o sistema.

Após a compilação do ficheiro `hello.c`, responsável pela criação do módulo, este foi carregado com sucesso no kernel. A verificação do carregamento e das mensagens associadas ao módulo foi realizada através do comando `dmesg`, que permite visualizar os registos do kernel.

A Figura 1 apresenta a saída do comando `dmesg`, onde é possível observar as mensagens geradas pelo módulo após o seu carregamento e remoção.

```
3831.666926] Hello World!
3832.664943] audit: type=1400 audit(1765881754.733:57853): apparmor="DENIED" operation="open" class="file" profile="snap.firefox.firefox" name="/proc/pressure/memory" pid=17454 comm="Memor
3837.465214] audit: type=1400 audit(1765881759.732:57854): apparmor="DENIED" operation="open" class="file" profile="snap.firefox.firefox" name="/proc/pressure/memory" pid=17454 comm="Memor
3842.667071] audit: type=1400 audit(1765881764.734:57855): apparmor="DENIED" operation="open" class="file" profile="snap.firefox.firefox" name="/proc/pressure/memory" pid=17454 comm="Memor
3847.652130] audit: type=1400 audit(1765881769.750:57856): apparmor="DENIED" operation="open" class="file" profile="snap.firefox.firefox" name="/proc/pressure/memory" pid=17454 comm="Memor
3850.676211] Bye-bye World!
```

Figura 1: Mensagens do kernel após o carregamento e remoção do módulo.

Questão 1B

Bloquear UDP para 8.8.8.8

Após a compilação do ficheiro `seedFilter.c` e o carregamento do respetivo módulo no kernel, foi possível testar se as regras da firewall se encontravam corretamente configuradas para bloquear tráfego UDP com destino ao endereço 8.8.8.8.

A seguinte Figura 2 demonstra o bloqueio do pedido.

```
carlos@carlos ~/Desktop/MCiber/SegurancaRedes/PL5/Labsetup/Files/kernel_module$ ./main & dig @8.8.8.8 www.example.com
;; communications error to 8.8.8.8#53: timed out
;; communications error to 8.8.8.8#53: timed out
;; communications error to 8.8.8.8#53: timed out

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @8.8.8.8 www.example.com
; (1 server found)
; global options: +cmd
; no servers could be reached
```

Figura 2: Dig @8.8.8.8 www.example.com

Netfilter hooks

O Netfilter disponibiliza vários hooks que permitem interceptar e processar pacotes em diferentes fases do seu percurso. Cada hook corresponde a um ponto específico onde podem ser aplicadas regras ou funções de filtragem.

- **NF_INET_PRE_ROUTING** — Pacotes que acabam de entrar na máquina, antes de qualquer decisão de encaminhamento.
- **NF_INET_LOCAL_IN** — Pacotes cujo destino é a própria máquina.
- **NF_INET_FORWARD** — Pacotes que atravessam a máquina para serem encaminhados para outro destino.
- **NF_INET_LOCAL_OUT** — Pacotes gerados localmente pela própria máquina.
- **NF_INET_POST_ROUTING** — Pacotes imediatamente antes de saírem da máquina.

Novos hooks na firewall

Nesta tarefa, foi necessário criar dois novos hooks para a firewall. O primeiro tem como objetivo bloquear pedidos de ping com destino ao host 10.9.0.1. Esta regra é implementada através de uma função que filtra os pacotes com base no protocolo ICMP e no endereço IP de destino pretendido.

```

1 unsigned int blockPing(void *priv, struct sk_buff *skb,
2                       const struct nf_hook_state *state)
3 {
4     struct iphdr *iph;
5     char ip[16] = "10.9.0.1";
6     u32 ip_addr;
7
8     if (!skb)
9         return NF_ACCEPT;
10
11     iph = ip_hdr(skb);
12     // Convert the IPv4 address from dotted decimal to 32-bit
13     // binary
14     in4_pton(ip, -1, (u8 *)&ip_addr, '\0', NULL);
15
16     if (iph->protocol == IPPROTO_ICMP)
17     {
18         if (iph->daddr == ip_addr)
19         {
20             printk(KERN_WARNING "*** Dropping %pI4 (ICMP)\n", &(
21                 iph->daddr));
22             return NF_DROP;
23         }
24     }
25     return NF_ACCEPT;
26 }
```

A Figura 3 seguinte demonstra que os pacotes ICMP com destino ao endereço especificado são efetivamente descartados.

```

root@621b0463b3dd:/# ping 10.9.0.1
PING 10.9.0.1 (10.9.0.1) 56(84) bytes of data.
^C
--- 10.9.0.1 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3058ms

```

Figura 3: Bloqueio de pacotes ICMP (ping) para o host 10.9.0.1.

O segundo hook criado tem como objetivo bloquear ligações TCP para o host 10.9.0.1 na porta 23, correspondente ao serviço Telnet. O código seguinte representa o mecanismo responsável por este bloqueio, filtrando os pacotes com base no protocolo, no endereço IP de destino e na porta utilizada.

```

1 unsigned int blockTelnet(void *priv, struct sk_buff *skb,
2                          const struct nf_hook_state *state)
3 {
4     struct iphdr *iph;
5     struct tcphdr *tcph;
6
7     u16 port = 23;
8     char ip[16] = "10.9.0.1";
9     u32 ip_addr;
10
11     if (!skb)
12         return NF_ACCEPT;
13
14     iph = ip_hdr(skb);
15     // Convert the IPv4 address from dotted decimal to 32-bit
16     // binary
17     in4_pton(ip, -1, (u8 *)&ip_addr, '\0', NULL);
18
19     if (iph->protocol == IPPROTO_TCP)
20     {
21         tcph = tcp_hdr(skb);
22         if (iph->daddr == ip_addr && ntohs(tcph->dest) == port)
23         {
24             printk(KERN_WARNING "*** Dropping %pI4 (TCP), port %d\n",
25                    &(iph->daddr), port);
26             return NF_DROP;
27         }
28     }
29     return NF_ACCEPT;
30 }

```

A Figura 4 seguinte ilustra o bloqueio das ligações TCP para o endereço e porta definidos, confirmando que os pacotes são descartados pela firewall.

```
root@621b0463b3dd:/# telnet 10.9.0.1
Trying 10.9.0.1...
^C
root@621b0463b3dd:/#
```

Figura 4: Bloqueio de ligações TCP (Telnet) para o host 10.9.0.1.

Questão 2A

Nesta tarefa, procedeu-se à definição de um conjunto de regras `iptables` no host router, com o objetivo de controlar o tráfego de rede.

A regra seguinte permite que o router receba pedidos de ping (ICMP echo-request):

```
iptables -A INPUT -p icmp --icmp-type echo-request -j ACCEPT
```

A regra abaixo permite que o router envie respostas aos pedidos de ping (ICMP echo-reply):

```
iptables -A OUTPUT -p icmp --icmp-type echo-reply -j ACCEPT
```

De seguida, foram definidas políticas por omissão restritivas, bloqueando todo o tráfego que não corresponda às regras anteriormente estabelecidas.

A política seguinte bloqueia todo o tráfego de entrada:

```
iptables -P INPUT DROP
```

Já esta política bloqueia todo o tráfego de saída:

```
iptables -P OUTPUT DROP
```

A Figura 5 ilustra o impacto das regras configuradas, evidenciando que os pacotes ICMP são permitidos, enquanto o tráfego TCP é bloqueado.

```

root@f61d3e1b8cbe:/# telnet 10.9.0.11
Trying 10.9.0.11...
telnet: Unable to connect to remote host: Connection timed out
root@f61d3e1b8cbe:/# telnet 10.9.0.11
Trying 10.9.0.11...
^C
root@f61d3e1b8cbe:/# ping 10.9.0.11
PING 10.9.0.11 (10.9.0.11) 56(84) bytes of data.
64 bytes from 10.9.0.11: icmp_seq=1 ttl=64 time=0.054 ms
64 bytes from 10.9.0.11: icmp_seq=2 ttl=64 time=0.087 ms
64 bytes from 10.9.0.11: icmp_seq=3 ttl=64 time=0.129 ms
64 bytes from 10.9.0.11: icmp_seq=4 ttl=64 time=0.090 ms
64 bytes from 10.9.0.11: icmp_seq=5 ttl=64 time=0.081 ms

```

Figura 5: Impacto das regras iptables no tráfego ICMP e TCP.

Questão 2B

De forma a garantir o cumprimento dos quatro requisitos propostos no exercício, foi inicialmente necessário identificar as interfaces de rede disponíveis no host router. A Figura 6 evidencia a existência de duas interfaces: **eth0**, correspondente à interface interna, e **eth1**, correspondente à interface externa.

```

root@f9fd5aa8d327:/# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0@if29: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether f6:70:2f:32:c8:82 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 192.168.60.11/24 brd 192.168.60.255 scope global eth0
        valid_lft forever preferred_lft forever
3: eth1@if30: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 0e:89:5b:32:2d:63 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.9.0.11/24 brd 10.9.0.255 scope global eth1
        valid_lft forever preferred_lft forever

```

Figura 6: Interfaces de rede disponíveis no router.

Após a identificação das interfaces, foi possível definir regras de firewall de acordo com o fluxo dos pacotes entre a rede interna e a rede externa. As regras apresentadas de seguida asseguram o cumprimento dos requisitos estabelecidos no enunciado.

A regra seguinte bloqueia pedidos ICMP do tipo echo-request que entrem

pela interface interna (**eth0**), impedindo que pedidos de ping provenientes da rede externa alcancem a rede interna:

```
iptables -A -i eth0 -p icmp --icmp-type echo-request -j DROP
```

A regra seguinte permite pedidos ICMP do tipo echo-request que entrem pela interface externa (**eth1**), possibilitando que pings da rede interna para a rede externa sejam efetuados:

```
iptables -A -i eth1 -p icmp --icmp-type echo-request -j ACCEPT
```

De modo a garantir que as respostas aos pings iniciados na rede interna não são bloqueadas, foi adicionada a seguinte regra, que permite pacotes ICMP do tipo echo-reply devolvidos à rede interna:

```
iptables -A -i eth0 -p icmp --icmp-type echo-reply -j ACCEPT
```

Por fim, foi definida uma política restritiva para o encaminhamento de pacotes entre redes, bloqueando todo o tráfego que não seja explicitamente permitido:

```
iptables -P FORWARD DROP
```

A Figura 7 apresenta o conjunto completo das regras de firewall ativas no router.

```
root@f9fd5aa8d327:/# iptables -A FORWARD -i eth0 -p icmp --icmp-type echo-request -j DROP
root@f9fd5aa8d327:/# iptables -A FORWARD -i eth1 -p icmp --icmp-type echo-request -j ACCEPT
root@f9fd5aa8d327:/# iptables -A FORWARD -i eth0 -p icmp --icmp-type echo-reply -j ACCEPT
root@f9fd5aa8d327:/# iptables -P FORWARD DROP
root@f9fd5aa8d327:/# iptables -L -n
Chain INPUT (policy ACCEPT)
target     prot opt source               destination

Chain FORWARD (policy DROP)
target     prot opt source               destination
DROP      icmp -- 0.0.0.0/0             0.0.0.0/0           icmptype 8
ACCEPT    icmp -- 0.0.0.0/0             0.0.0.0/0           icmptype 8
ACCEPT    icmp -- 0.0.0.0/0             0.0.0.0/0           icmptype 0

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
```

Figura 7: Conjunto de regras **iptables** ativas no router.

As figuras seguintes demonstram a preservação dos requisitos definidos no exercício, evidenciando o comportamento esperado da firewall em diferentes cenários de comunicação.

```
root@f61d3e1b8cbe:/# ping 192.168.60.5
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
□
```

Figura 8: Bloqueio de pedidos de ping da rede externa para a rede interna.

```
root@f61d3e1b8cbe:/# ping 10.9.0.11
PING 10.9.0.11 (10.9.0.11) 56(84) bytes of data.
64 bytes from 10.9.0.11: icmp_seq=1 ttl=64 time=0.165 ms
64 bytes from 10.9.0.11: icmp_seq=2 ttl=64 time=0.124 ms
64 bytes from 10.9.0.11: icmp_seq=3 ttl=64 time=0.130 ms
64 bytes from 10.9.0.11: icmp_seq=4 ttl=64 time=0.082 ms
□
```

Figura 9: Pedido de ping da rede externa para o router.

```
root@fb9eb63c58bb:/# ping 10.9.0.5
PING 10.9.0.5 (10.9.0.5) 56(84) bytes of data.
64 bytes from 10.9.0.5: icmp_seq=1 ttl=63 time=0.108 ms
64 bytes from 10.9.0.5: icmp_seq=2 ttl=63 time=0.115 ms
64 bytes from 10.9.0.5: icmp_seq=3 ttl=63 time=0.135 ms
□
```

Figura 10: Pedido de ping da rede interna para a rede externa.

```
root@fb9eb63c58bb:/# telnet 10.9.0.5
Trying 10.9.0.5...
□
```

Figura 11: Bloqueio de ligações Telnet da rede interna para a rede externa.

Questão 2C

De forma a garantir o cumprimento dos requisitos desta tarefa, foram definidas regras específicas no `iptables`.

A primeira regra permite o encaminhamento de ligações TCP destinadas à porta 23, provenientes da rede externa e com destino ao host interno 192.168.60.5:

```
iptables -A FORWARD -i eth0 -d 192.168.60.5 -p tcp \
--dport 23 -j ACCEPT
```

A segunda regra permite o encaminhamento dos pacotes de resposta associados às ligações Telnet estabelecidas, provenientes do host interno 192.168.60.5 e com origem na porta 23:

```
iptables -A FORWARD -i eth1 -s 192.168.60.5 -p tcp \
--sport 23 -j ACCEPT
```

Por fim, foi definida uma política restritiva para a cadeia FORWARD, bloqueando todo o tráfego que não seja explicitamente permitido pelas regras anteriores:

```
iptables -P FORWARD DROP
```

As figuras seguintes ilustram o resultado da aplicação destas regras.



```
root@f61d3e1b8cbe:/# telnet 192.168.60.7
Trying 192.168.60.7...
^C
root@f61d3e1b8cbe:/# telnet 192.168.60.5
Trying 192.168.60.5...
Connected to 192.168.60.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
fb9eb63c58bb login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 6.14.0-36-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Wed Dec 17 21:46:54 UTC 2025 from 10.9.0.5 on pts/1
seed@fb9eb63c58bb:~$
```

Figura 12: Ligação Telnet da rede externa para a rede interna, permitida apenas para o host 192.168.60.5.

```

root@fb9eb63c58bb:/# telnet 192.168.60.6
Trying 192.168.60.6...
Connected to 192.168.60.6.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
991e1337b245 login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 6.14.0-36-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Wed Dec 17 20:57:30 UTC 2025 from d80c9096a32b_host1-192.168.60.5.net-192.168.60.0 on pts/1

```

Figura 13: Ligações Telnet entre hosts internos.

```

root@fb9eb63c58bb:/# telnet 10.9.0.5
Trying 10.9.0.5...
telnet: Unable to connect to remote host: Connection timed out

```

Figura 14: Bloqueio de ligações Telnet da rede interna para a rede externa.

Questão 3A

ICMP

No caso do protocolo ICMP, o estado de cada comunicação é mantido na tabela de conntrack durante aproximadamente 30 segundos após o último pacote observado. Caso não exista tráfego adicional durante esse intervalo, a entrada é automaticamente removida.

Cada novo pedido de ping renova o temporizador associado à entrada existente. O mecanismo de conntrack identifica os pacotes ICMP com base no seu tipo, capturando pedidos echo-request e respectivas respostas echo-reply, bem como os endereços IP de origem e de destino.

A Figura 15 apresenta uma entrada de conntrack correspondente a tráfego ICMP.

```

root@f9fd5aa8d327:/# conntrack -L
icmp    1 29 src=10.9.0.5 dst=192.168.60.5 type=8 code=0 id=3 src=192.168.60.5 dst=10.9.0.5 type=0 code=0 id=3 mark=0 use=1
icmp    1 11 src=10.9.0.5 dst=192.168.60.5 type=8 code=0 id=2 src=192.168.60.5 dst=10.9.0.5 type=0 code=0 id=2 mark=0 use=1
conntrack v1.4.5 (conntrack-tools): 2 flow entries have been shown.

```

Figura 15: Entrada de conntrack para tráfego ICMP.

UDP

No exemplo observado, o porta de origem corresponde a `sport = 52085`, associado ao cliente, enquanto a porta de destino corresponde a `dport = 9090`, associado ao servidor.

O estado `[UNREPLIED]` indica que o router ainda não detetou qualquer pacote de resposta proveniente do destino para a origem. Tal comportamento é expectável em comunicações UDP, uma vez que este protocolo não é orientado à ligação.

À semelhança do ICMP, o tempo de vida da entrada UDP na tabela de `conntrack` é, por omissão, de aproximadamente 30 segundos.

A Figura 16 ilustra uma entrada correspondente a tráfego UDP.

```
root@f9fd5aa8d327:/# conntrack -L
udp      17 6 src=10.9.0.5 dst=192.168.60.5 sport=52085 dport=9090 [UNREPLIED] src=192.168.60.5 dst=10.9.0.5 sport=9090 dport=52085 mark=0 use=1
conntrack v1.4.5 (conntrack-tools): 1 flow entries have been shown.
```

Figura 16: Entrada de `conntrack` para tráfego UDP.

TCP

No caso do protocolo TCP, as entradas na tabela de `conntrack` possuem um tempo de vida superior, tipicamente em torno de 2 minutos.

O estado `[ASSURED]` indica que a ligação TCP foi completamente estabelecida, ou seja, que o three-way handshake foi concluído com sucesso e que já ocorreu tráfego em ambos os sentidos da comunicação.

A Figura 17 apresenta uma entrada de `conntrack` correspondente a uma ligação TCP estabelecida.

```
root@f9fd5aa8d327:/# conntrack -L
tcp      6 431995 ESTABLISHED src=10.9.0.5 dst=192.168.60.5 sport=48836 dport=9090 src=192.168.60.5 dst=10.9.0.5 sport=9090 dport=48836 [ASSURED] mark=0 use=1
conntrack v1.4.5 (conntrack-tools): 1 flow entries have been shown.
```

Figura 17: Entrada de `conntrack` para tráfego TCP.

Questão 3B

Nesta tarefa, foram adicionadas regras de firewall com recurso a connection tracking, permitindo apenas tráfego nos estados `NEW`, `ESTABLISHED` e `RELATED`. Esta abordagem possibilita a implementação de uma firewall stateful, capaz de acompanhar o estado das ligações ativas.

A regra seguinte permite o estabelecimento de novas ligações Telnet provenientes da rede externa, com destino ao host interno `192.168.60.5`:

```
iptables -A FORWARD -p tcp -i eth0 -d 192.168.60.5 --dport 23 --syn \
-m conntrack --ctstate NEW -j ACCEPT
```

A regra seguinte permite o estabelecimento de novas ligações Telnet iniciadas a partir da rede interna com destino à rede externa:

```
iptables -A FORWARD -p tcp -i eth1 --dport 23 --syn \
-m conntrack --ctstate NEW -j ACCEPT
```

De forma a garantir o correto funcionamento das ligações já estabelecidas, foi adicionada uma regra que permite pacotes associados a ligações existentes ou relacionadas:

```
iptables -A FORWARD -p tcp -m conntrack --ctstate RELATED, \
ESTABLISHED -j ACCEPT
```

Todo o restante tráfego TCP que não corresponda às regras anteriores é explicitamente bloqueado pela seguinte regra:

```
iptables -A FORWARD -p tcp -j DROP
```

Por fim, foi definida a política por defeito da cadeia FORWARD como ACCEPT, aplicando-se apenas aos pacotes que não correspondam a nenhuma regra configurada:

```
iptables -P FORWARD ACCEPT
```

Sem o recurso a connection tracking, seria necessário permitir explicitamente as portas de resposta, o que tornaria as regras mais complexas, difíceis de manter e potencialmente menos seguras. Deste modo, a utilização de uma firewall stateful, baseada em conntrack, apresenta vantagens claras em termos de segurança e facilidade de gestão. Em contraste, uma firewall stateless consome menos recursos, mas exige uma configuração mais detalhada e está mais sujeita a erros.

Questão 4

Nesta tarefa, foi utilizado o módulo limit do iptables este mecanismo permite controlar a taxa de pacotes permitidos, funcionando como uma barreira contra o tráfego excessivo.

Foram aplicadas as seguintes regras no router:

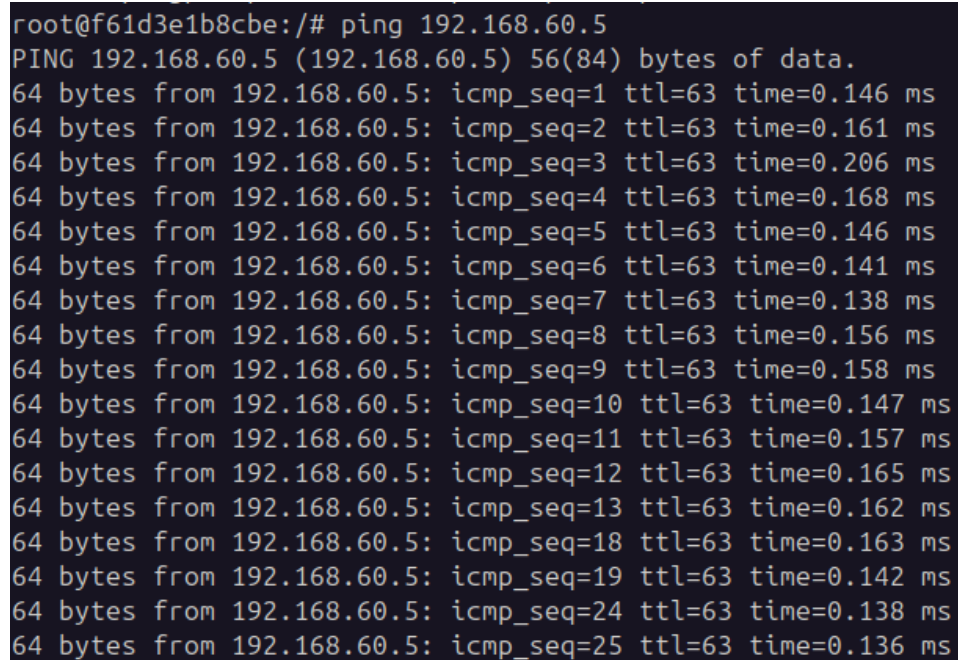
```
iptables -A FORWARD -s 10.9.0.5 -m limit \
--limit 10/minute --limit-burst 5 -j ACCEPT
```

```
iptables -A FORWARD -s 10.9.0.5 -j DROP
```

A primeira regra permite que pacotes provenientes do endereço 10.9.0.5 sejam encaminhados, desde que não excedam uma taxa média de 10 pacotes por minuto, com um pico inicial máximo de 5 pacotes. A segunda regra bloqueia todo o restante tráfego proveniente desse mesmo host que ultrapasse o limite definido.

Ao efetuar pedidos de ping a partir do host 10.9.0.5 para o endereço 192.168.60.5, observou-se que, com ambas as regras ativas, a partir do pacote identificado como `icmp_seq=13` começam a surgir atrasos significativos nas respostas ICMP. Este comportamento indica que os pacotes adicionais estão a ser atrasados ou descartados devido à limitação imposta pelo módulo `limit`.

A Figura 18 ilustra o comportamento observado durante o teste de limitação de tráfego ICMP.



```
root@f61d3e1b8cbe:/# ping 192.168.60.5
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
64 bytes from 192.168.60.5: icmp_seq=1 ttl=63 time=0.146 ms
64 bytes from 192.168.60.5: icmp_seq=2 ttl=63 time=0.161 ms
64 bytes from 192.168.60.5: icmp_seq=3 ttl=63 time=0.206 ms
64 bytes from 192.168.60.5: icmp_seq=4 ttl=63 time=0.168 ms
64 bytes from 192.168.60.5: icmp_seq=5 ttl=63 time=0.146 ms
64 bytes from 192.168.60.5: icmp_seq=6 ttl=63 time=0.141 ms
64 bytes from 192.168.60.5: icmp_seq=7 ttl=63 time=0.138 ms
64 bytes from 192.168.60.5: icmp_seq=8 ttl=63 time=0.156 ms
64 bytes from 192.168.60.5: icmp_seq=9 ttl=63 time=0.158 ms
64 bytes from 192.168.60.5: icmp_seq=10 ttl=63 time=0.147 ms
64 bytes from 192.168.60.5: icmp_seq=11 ttl=63 time=0.157 ms
64 bytes from 192.168.60.5: icmp_seq=12 ttl=63 time=0.165 ms
64 bytes from 192.168.60.5: icmp_seq=13 ttl=63 time=0.162 ms
64 bytes from 192.168.60.5: icmp_seq=18 ttl=63 time=0.163 ms
64 bytes from 192.168.60.5: icmp_seq=19 ttl=63 time=0.142 ms
64 bytes from 192.168.60.5: icmp_seq=24 ttl=63 time=0.138 ms
64 bytes from 192.168.60.5: icmp_seq=25 ttl=63 time=0.136 ms
```

Figura 18: Efeito da limitação de tráfego ICMP utilizando o módulo `limit`.

Questão 5

Nesta tarefa utilizamos o módulo `statistics` do `iptables` de forma a implementar um `load balancer` capaz de distribuir, por 3 servidores UDP, pacotes UDP recebidos pela rede interna.

Este módulo fornece 2 modos de fazer `load balancing`: o `nth mode` (`round-robin`) e o `random mode`. Apesar de ambos os modos servirem

para **load balancing**, o forma como estes distribuem os pacotes é diferente, sendo o **random mode** baseado em probabilidades e o **nth mode** num algoritmo **round-robin**.

Nth mode

Para implementar o **nth mode**, usamos um conjunto de 3 regras para que os servidores UDP recebessem o mesmo número de pacotes.

```
root@c278eae730cd:/# iptables -t nat -A PREROUTING -p udp --dport 8080 \
> -m statistic --mode nth --every 3 --packet 0 \
> -j DNAT --to-destination 192.168.60.5:8080
root@c278eae730cd:/# iptables -t nat -A PREROUTING -p udp --dport 8080 -m statistic --mode nth --every 2 --packet 0 -j DNAT --to-destination 192.168.60.6:8080
root@c278eae730cd:/# iptables -t nat -A PREROUTING -p udp --dport 8080 -j DNAT --to-destination 192.168.60.7:8080
```

Figura 19: Regras usadas para distribuir pacotes de forma uniforme.

- **1ª Regra:**

```
iptables -t nat -A PREROUTING -p udp --dport 8080 \
-m statistic --mode nth --every 3 --packet 0 \
-j DNAT --to-destination 192.168.60.5:8080
```

Esta regra faz com que, a cada 3 pacotes recebidos pelo **router**, o primeiro pacote (**packet 0**) é reencaminhado para o servidor UDP com ip 192.168.60.5.

- **2ª Regra:**

```
iptables -t nat -A PREROUTING -p udp --dport 8080 \
-m statistic --mode nth --every 2 --packet 0 \
-j DNAT --to-destination 192.168.60.6:8080
```

Para reencaminhar para o ip 192.168.60.6, alteramos a regra para reencaminhar o primeiro pacote em cada 2. Isto deve-se a forma como funcionam as regras na **iptables**. Como as regras são aplicadas em ordem *top-down*, apenas 2 dos 3 pacotes enviados chegam a ser avaliados pela 2ª regra, pelo que é preciso mudar o parâmetro **—every** para 2.

- 3ª Regra:

```
iptables -t nat -A PREROUTING -p udp --dport 8080 \
-j DNAT --to-destination 192.168.60.7:8080
```

A última regra é a mais simples sendo apenas necessário reencaminhar todos os pacotes recebidos por esta regra para o ip 192.168.60.7. Isto deve-se ao já referido na justificação da regra anterior. Sendo que a regra anterior só fica com 1 dos 2 pacotes que recebe, o pacote restante é diretamente reencaminhado para o último servidor.



```
[12/21/25]seed@VM:~/PL5/Labsetup$ docksh 94
root@94608b2131d8:/# nc -luk 8080
hello
```

Figura 20: Servidor com ip 192.168.60.5 a receber 1 dos 3 pacotes enviados



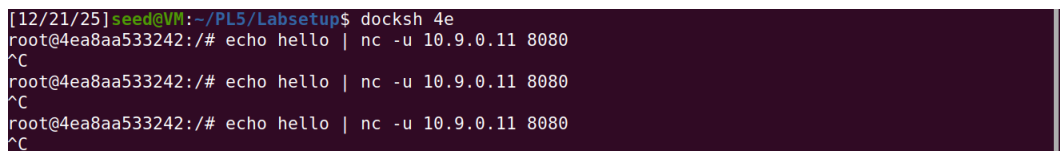
```
[12/21/25]seed@VM:~/PL5/Labsetup$ docksh 85
root@855f2c2e7281:/# nc -luk 8080
hello
```

Figura 21: Servidor com ip 192.168.60.6 a receber 1 dos 3 pacotes enviados



```
[12/21/25]seed@VM:~/PL5/Labsetup$ docksh 3d
root@3dd64009c682:/# nc -luk 8080
hello
```

Figura 22: Servidor com ip 192.168.60.7 a receber 1 dos 3 pacotes enviados



```
[12/21/25]seed@VM:~/PL5/Labsetup$ docksh 4e
root@4ea8aa533242:/# echo hello | nc -u 10.9.0.11 8080
^C
root@4ea8aa533242:/# echo hello | nc -u 10.9.0.11 8080
^C
root@4ea8aa533242:/# echo hello | nc -u 10.9.0.11 8080
^C
```

Figura 23: Máquina com ip 10.9.0.5 a enviar pacotes UDP para o router

Random mode

Para implementar o **random mode**, usamos um conjunto de 3 regras para que os servidores UDP recebessem aproximadamente o mesmo número de pacotes.

```
root@c278eae730cd:/# iptables -t nat -A PREROUTING -p udp --dport 8080 -m statistic --mode random --probability 0.3333333333 -j DNAT --to-destination 192.168.60.5:8080
root@c278eae730cd:/# iptables -t nat -A PREROUTING -p udp --dport 8080 -m statistic --mode random --probability 0.5 -j DNAT --to-destination 192.168.60.6:8080
root@c278eae730cd:/# iptables -t nat -A PREROUTING -p udp --dport 8080 -j DNAT --to-destination 192.168.60.7:8080
```

Figura 24: Regras usadas para distribuir pacotes de forma uniforme.

- **1ª Regra:**

```
iptables -t nat -A PREROUTING -p udp --dport 8080 \
-m statistic --mode random --probability 0.3333333333 \
-j DNAT --to-destination 192.168.60.5:8080
```

Esta regra faz com que cada pacote recebido pelo **router** tenha uma probabilidade de $\frac{1}{3}$ (0.3333333333) de ser reencaminhado para o servidor UDP com ip 192.168.60.5.

- **2ª Regra:**

```
iptables -t nat -A PREROUTING -p udp --dport 8080 \
-m statistic --mode random --probability 0.5 \
-j DNAT --to-destination 192.168.60.6:8080
```

De forma análoga ao que acontece no **nth mode**, a segunda regra recebe um pacote em $\frac{2}{3}$ das vezes, logo nesta regra temos de alterar a probabilidade para $\frac{1}{2}$ de forma a que o servidor com ip 192.168.60.6 tenha uma probabilidade de $\frac{1}{3}$ de receber um pacote.

- **3ª Regra:**

```
iptables -t nat -A PREROUTING -p udp --dport 8080 \  
-j DNAT --to-destination 192.168.60.7:8080
```

Como a última regra só recebe um pacote em $\frac{1}{3}$ das vezes, podemos re-encaminhar o pacote para o servidor com ip 192.168.60.7 com uma probabilidade de 1.



```
root@94608b2131d8:/# nc -luk 8080  
hello  
hello  
hello  
hello  
hello  
█
```

Figura 25: Servidor com ip 192.168.60.5 a receber pacotes UDP



```
root@855f2c2e7281:/# nc -luk 8080  
hello  
hello  
hello  
hello  
hello  
hello  
█
```

Figura 26: Servidor com ip 192.168.60.6 a receber pacotes UDP



```
root@3dd64009c682:/# nc -luk 8080  
hello  
hello  
█
```

Figura 27: Servidor com ip 192.168.60.7 a receber pacotes UDP

[illegible]

Figura 28: Máquina com ip 10.9.0.5 a enviar pacotes UDP para o router

Como este modo usa probabilidades, o número de pacotes recebido por cada servidor pode não ser exatamente igual, como podemos ver nas imagens apresentadas acima.